



Context Course documentation

Quiz 2: Building and Distributing Plugins ▾



Quiz 2: Building and Distributing Plugins

[Copy page](#) ▾

Test your understanding of plugin development, distribution, and best practices.

Question 1: Where does plugin.json belong?

- ☐ Claude Code and Codex use the same plugin.json schema
- ☐ OpenCode also uses plugin.json to load local plugins
- ☒ In Claude Code and Codex, plugin.json defines the plugin's identity and tells the agent where to find bundled components

Correct! Correct! On manifest-first platforms, the manifest identifies the plugin and points to bundled components such as skills, MCP config, or app integrations.

- ☐ The plugin.json should be placed at the root of the plugin directory

[Submit](#)**You got all the answers!**

Question 2: What does the manifest-first plugin structure look like?

- ☒ A directory with .claude-plugin/ or .codex-plugin/ manifest directory, skills/ folder, optional .mcp.json, and README

Correct! Correct! That's the manifest-first structure used by Claude Code and Codex.

- ☐ Just a skills/ folder with SKILL.md files — no manifest needed
- ☐ A single file named manifest.json containing everything
- ☐ A root manifest.json with an openapi.json spec

[Ask HuggingChat](#)

You got all the answers!

Question 3: How should plugin permissions be designed?

- ☒ Request only the permissions you actually need; document why each is needed; follow the principle of least privilege

Correct! Correct! Minimal, documented permissions build user trust. Both platforms emphasize security-conscious plugin design.

- ☐ Request all possible permissions upfront to maximize future flexibility
- ☐ Hide permission details in the README so users don't worry about them
- ☐ Permissions don't matter much; focus only on functionality

You got all the answers!

Question 4: How do you test plugins locally?

- ☒ Test with `claude --plugin-dir` for Claude Code; create a local `marketplace.json` entry for Codex

Correct! Correct! Claude Code uses `--plugin-dir` for local development. Codex uses local marketplace entries for unpublished plugins.

- ☐ You must publish to the marketplace before you can test your plugin
- ☐ Run `/plugin validate` to check your plugin structure
- ☐ Testing isn't necessary; just publish and fix issues later

You got all the answers!

Question 5: How should secrets be handled?

- ☐ Always hardcode API keys in the manifest for convenience
- ☒ Use environment variables for secrets, reference supported, and document how to set them

 Ask HuggingChat

Correct! Correct! This is the secure approach. Environment variables keep secrets out of version control and plugin code.

- ☒ Prompt users for API keys at runtime
- ☒ Store API keys in a config file that users commit to git

Submit

You got all the answers!

Summary

If you got 4-5 correct, you're ready to build and distribute plugins responsibly. If not, review the build and usage lessons before publishing anything.

Key Takeaways

- Keep plugin structure explicit: manifest, bundled components, and documentation each have a clear place
- Ask for the minimum permissions you need and explain why
- Test locally before publishing, and keep secrets in environment variables rather than plugin files

Next Steps

You've finished Unit 3: Plugins. You now know how to package reusable agent behavior and distribute it safely. Next up is Unit 4, where those components become building blocks for multi-agent workflows.

</> [Update on GitHub](#)

Ask HuggingChat



← Using Plugins

Unit 4: Subagents →



Ask HuggingChat

